

Applied Data Science

Practical Exam Guide

All 10 Experiments — Code + Explanation

Experiment Index

No.	Experiment Title	Dataset	COs
1	Descriptive Statistics	supermarket_sales	CO1, CO2
2	Data Cleaning & Imputation	loan_data_set	CO1, CO2
3	Data Visualization	supermarket_sales	CO1, CO2
4	Supervised Model Metrics	breast_cancer / regdata	CO1, CO3
5	Unsupervised Model Metrics	iris	CO1, CO3
6	Time Series Forecasting (ARIMA)	supermarket_sales	CO1, CO3
7	Outlier Detection	iris	CO1, CO3
8	SMOTE — Class Imbalance	Churn_Modelling	CO1, CO2
9	Hypothesis Testing (Z/T/ANOVA)	supermarket_sales	CO1, CO2
10	Case Study	Any	CO1, CO2, CO3

Aim

Explore descriptive statistics on the supermarket_sales dataset — central tendency, spread, shape, and correlation.

Concepts at a Glance

Mean	Sum of all values divided by count. Sensitive to outliers.
Median	Middle value when sorted. Robust to outliers.
Mode	Most frequently occurring value. Best for categorical data.
Trimmed Mean	Mean after removing top & bottom 10% — less affected by outliers.
Variance / Std Dev	Measures how spread out values are from the mean.
Skewness	Asymmetry of the distribution (+ = right tail, - = left tail).
Kurtosis	Peakedness. High kurtosis = heavy tails (more outliers likely).
Correlation	Linear relationship between two variables (range: -1 to +1).

Full Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv('supermarket_sales - Sheet1.csv')

# ---- BASIC INFO ----
print(df.head())          # First 5 rows
print(df.info())          # Dtypes + null counts
print(df.describe())      # Mean, Std, Min, Max, Quartiles for numeric cols

# ---- CENTRAL TENDENCY ----
print(df.mean(numeric_only=True))  # Mean of all numeric columns
print(df.median(numeric_only=True)) # Median
# Mode for categorical columns
for col in ['Product line', 'City', 'Payment', 'Customer type', 'Gender']:
    print(f'Mode of {col}:', df[col].mode()[0])
```

```

# ---- TRIMMED MEAN ----
print('Trimmed Mean of Total:', stats.trim_mean(df['Total'], 0.1))
print('Regular Mean of Total:', df['Total'].mean())
# Inference: if trimmed mean << mean, outliers are inflating the mean

# ---- SPREAD ----
print(df.var(numeric_only=True))      # Variance
print(df.std(numeric_only=True))      # Standard Deviation
print(df.sem(numeric_only=True))      # Standard Error of Mean

# ---- SHAPE ----
print(df.skew(numeric_only=True))     # Skewness
print(df['Total'].kurtosis())          # Kurtosis of Total column

# ---- SUM OF SQUARES ----
sos = sum(val**2 for val in df['Total'])
print('Sum of Squares:', sos)

# ---- FREQUENCY ----
print(df['Product line'].value_counts())

# ---- CORRELATION MATRIX ----
print(df.corr(numeric_only=True))

# ---- SCATTER PLOT ----
plt.scatter(df['Tax 5%'], df['Unit price'], c='blue')
plt.xlabel('Tax 5%'); plt.ylabel('Unit price')
plt.title('Tax vs Unit Price'); plt.show()

# ---- BOX PLOT ----
data_box = df[['Tax 5%', 'gross income', 'Rating']]
data_box.plot(kind='box', title='Boxplot'); plt.show()

# ---- DIST PLOT ----
sns.histplot(df['Total'], kde=True)
plt.title('Distribution of Total'); plt.show()

```

 **Tip:** In the exam, if asked to compare trimmed mean vs mean: if they are close, there are few outliers. If trimmed mean is significantly lower, high-value outliers are pulling the mean up.

Sample Question (from your exam paper)

Q1 (diamonds.csv) — same concepts apply to any dataset:

- `describe()` gives mean, std, min, max, quartiles for all numeric cols
- `df['price'].mean() / .median() / .mode()[0]`
- Boxplot: `plt.boxplot(df['depth'])` — whiskers show upper/lower limits
- Heatmap: `sns.heatmap(df.corr(numeric_only=True), annot=True)` — look for values > 0.7
- Bubble chart: `px.scatter(df, x='x', y='y', size='z')` using plotly express

Aim

Handle missing values in the loan_data_set.csv using various imputation techniques, and encode categorical features.

Imputation Techniques

Technique	When to use	Drawback
Mean Imputation	Numeric cols, normally distributed data	Reduces variance
Median Imputation	Numeric cols with outliers	Ignores data distribution
Mode Imputation	Categorical or skewed data	Creates imbalance
Regression Imputation	When correlated features exist	Complex, overfitting risk
Random Sample	When distribution needs preserving	Introduces randomness
Frequent Category	Categorical missing values	Same as mode

Full Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OrdinalEncoder
from sklearn.linear_model import LinearRegression

df = pd.read_csv('loan_data_set.csv')

# ---- FIND MISSING VALUES ----
na_variables = [var for var in df.columns if df[var].isnull().mean() > 0]
print('Columns with nulls:', na_variables)
print(df.isnull().sum())      # Count of nulls per column
print(df.isnull().mean()*100) # Percentage of nulls per column

# ---- MEAN IMPUTATION ----
df_mean = df.copy()
missing_col = ['LoanAmount']
for col in missing_col:
    df_mean[col].fillna(df_mean[col].mean(), inplace=True)
```

```

# ---- MEDIAN IMPUTATION ----
df_median = df.copy()
for col in missing_col:
    df_median[col].fillna(df_median[col].median(), inplace=True)

# ---- MODE IMPUTATION ----
df_mode = df.copy()
for col in missing_col:
    df_mode[col].fillna(df_mode[col].mode()[0], inplace=True)

# ---- FREQUENT CATEGORY IMPUTATION (Categorical) ----
df_freq = df.copy()
df_freq['Gender'].fillna(df_freq['Gender'].mode()[0], inplace=True)

# ---- RANDOM SAMPLE IMPUTATION ----
df_rand = df.copy()
random_sample = df_rand['LoanAmount'].dropna().sample(
    df_rand['LoanAmount'].isnull().sum(), random_state=0)
random_sample.index = df_rand[df_rand['LoanAmount'].isnull()].index
df_rand.loc[df_rand['LoanAmount'].isnull(), 'LoanAmount'] = random_sample

# ---- REGRESSION IMPUTATION ----
lr = LinearRegression()
df_reg = df[['CoapplicantIncome', 'LoanAmount']].copy()
traindf = df_reg[df_reg['LoanAmount'].notna()]
testdf = df_reg[df_reg['LoanAmount'].isna()]
lr.fit(traindf[['CoapplicantIncome']], traindf['LoanAmount'])
predicted = lr.predict(testdf[['CoapplicantIncome']])
df_reg.loc[df_reg['LoanAmount'].isna(), 'LoanAmount'] = predicted

# ---- ORDINAL ENCODING (Categorical → Numeric) ----
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder()
df_encoded = df.copy()
cat_cols = df_encoded.select_dtypes(include='object').columns
df_encoded[cat_cols] = oe.fit_transform(df_encoded[cat_cols].astype(str))
print(df_encoded.head())

```

 **Note:** In the exam, always check for nulls first with `df.isnull().sum()` before applying any imputation. State why you chose a particular method in your viva.

Aim

Visualize supermarket_sales data using scatter, box, histogram, pie, density, bubble, parallel coordinates, and more.

Plot Types Quick Reference

Chart Type	Purpose	Library
Scatter Plot	Relationship between 2 numeric variables	matplotlib
Box Plot	Distribution + outliers for numeric columns	pandas / matplotlib
Histogram	Frequency distribution of 1 variable	pandas
Dist Plot	Smoothed distribution (KDE + histogram)	seaborn
Pie Chart	Proportion of categories	matplotlib
Density Plot	Continuous distribution estimate	pandas
Joint Plot	Scatter + marginal distributions	seaborn
Pair Plot	All pairwise relationships in a dataset	seaborn
Bubble Chart	Scatter with a 3rd size dimension	plotly
Line Chart	Trends over time/sequence	plotly
Parallel Coords	Multi-dimensional feature comparison	plotly
Andrews Curves	Pattern detection in multi-dim data	pandas
Rug Plot	1D data distribution along an axis	seaborn
Heatmap	Correlation matrix as colour grid	plotly / seaborn

Full Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

df = pd.read_csv('supermarket_sales - Sheet1.csv')

# ---- SCATTER PLOT ----
plt.scatter(df['Tax 5%'], df['Unit price'], c='blue')
plt.xlabel('Tax 5%'); plt.ylabel('Unit price'); plt.show()
```

```

# ---- BOX PLOT ----
data_box = df[['Tax 5%', 'gross income', 'Rating']]
data_box.plot(kind='box', title='Boxplot'); plt.show()

# ---- HISTOGRAM ----
df['Rating'].hist(); plt.title('Rating Distribution'); plt.show()

# ---- DIST PLOT (seaborn) ----
sns.histplot(df['Total'], kde=True); plt.show()

# ---- PIE CHART ----
lst = df['Product line'].unique()
t = df['Product line'].value_counts().values
plt.pie(t, labels=lst, autopct='%1.1f%%', shadow=True); plt.show()

# ---- DENSITY PLOT ----
df['Rating'].plot.density(color='green')
plt.title('Density Plot - Rating'); plt.show()

# ---- JOINT PLOT ----
sns.jointplot(x='Total', y='Tax 5%', data=df); plt.show()

# ---- PAIR PLOT ----
sns.pairplot(df); plt.show()

# ---- RUG PLOT ----
plt.figure(figsize=(15,5))
sns.rugplot(data=df, x='Total'); plt.show()

# ---- COLUMN/BAR CHART ----
df1 = df.head(10)
plt.bar(df1['Gender'], df1['Total'])
plt.xlabel('Gender'); plt.ylabel('Total'); plt.show()

# ---- PLOTLY LINE CHART ----
df1 = df.head(15)
fig = px.line(df1, x='Date', y='Total', color='City'); fig.show()

# ---- BUBBLE CHART ----
fig = px.scatter(df1, x='Total', y='Tax 5%', size='Quantity',
                color='City', hover_name='Product line',
                log_x=True, size_max=60); fig.show()

# ---- PARALLEL COORDINATES ----
df_s = df.sample(n=100)

```



```
fig = px.parallel_coordinates(df_s, color='Total',
                             dimensions=['Quantity','Unit price','Rating']); fig.show()

# ---- ANDREWS CURVES ----
df_ac = df[['Quantity','Total','Rating']].sample(n=50)
pd.plotting.andrews_curves(df_ac, 'Rating'); plt.show()

# ---- SCATTER MATRIX ----
pd.plotting.scatter_matrix(df); plt.show()

# ---- HEATMAP (using plotly) ----
fig = px.imshow(df[['Unit price','Quantity','Tax 5%','Total','Rating']].corr())
fig.show()
```

Aim

Implement classification (Decision Tree on breast_cancer) and regression (Linear Regression) models, then evaluate them using appropriate metrics.

Part A — Classification Metrics

 **Note:** The confusion matrix gives you TP, TN, FP, FN — everything else is derived from these four numbers.

Metric	Formula	Range / Ideal
Accuracy	$(TP+TN) / (TP+TN+FP+FN)$	0–1, higher=better
Error Rate	$(FP+FN) / \text{Total}$	0–1, lower=better
Precision	$TP / (TP+FP)$	0–1, higher=better
Sensitivity/Recall	$TP / (TP+FN)$	0–1, higher=better
Specificity	$TN / (TN+FP)$	0–1, higher=better
F1 Score	$2 * (\text{Prec} * \text{Sens}) / (\text{Prec} + \text{Sens})$	0–1, higher=better
FPR (False +ve Rate)	$1 - \text{Specificity}$	0–1, lower=better
FNR (False -ve Rate)	$1 - \text{Sensitivity}$	0–1, lower=better
Geometric Mean (GM)	$\text{sqrt}(\text{Sensitivity} * \text{Specificity})$	0–1, higher=better
Power	$1 - \text{FNR} \quad (= \text{Sensitivity})$	0–1, higher=better

Classification Code

```
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, roc_curve, roc_auc_score
import matplotlib.pyplot as plt
import math

dataset = datasets.load_breast_cancer()
X, y = dataset.data, dataset.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
```

```

y_pred = clf.predict(X_test)

# ---- CONFUSION MATRIX ----
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
print(f'TP={tp}, TN={tn}, FP={fp}, FN={fn}')

# ---- ALL METRICS ----
acc      = (tp + tn) / (tp + tn + fp + fn)
err      = (fp + fn) / (tp + tn + fp + fn)
prec     = tp / (tp + fp)
sens     = tp / (tp + fn)   # Sensitivity / Recall
spec     = tn / (tn + fp)   # Specificity
f1       = 2 * sens * prec / (prec + sens)
fpr      = 1 - spec
fnr      = 1 - sens
power    = 1 - fnr
gm       = math.sqrt(sens * spec)
roc_val  = math.sqrt(sens**2 + spec**2) / math.sqrt(2)

print(f'Accuracy      : {acc:.4f}')
print(f'Error Rate    : {err:.4f}')
print(f'Precision     : {prec:.4f}')
print(f'Sensitivity    : {sens:.4f}')
print(f'Specificity    : {spec:.4f}')
print(f'F1 Score      : {f1:.4f}')
print(f'FPR          : {fpr:.4f}')
print(f'FNR          : {fnr:.4f}')
print(f'Power        : {power:.4f}')
print(f'Geo Mean     : {gm:.4f}')

# ---- ROC CURVE ----
fpr_arr, tpr_arr, _ = roc_curve(y_test, y_pred)
print('AUC Score:', roc_auc_score(y_test, y_pred))
plt.figure(figsize=(8,8))
plt.plot(fpr_arr, tpr_arr, label='Decision Tree')
plt.plot([0,1],[0,1], '--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('ROC Curve'); plt.legend(); plt.show()

```

Part B — Regression Metrics

Metric	Meaning	Ideal
MSE	Mean Squared Error — penalizes large errors heavily	Close to 0
RMSE	Root of MSE — in same units as target	Close to 0
MAE	Mean Absolute Error — average magnitude	Close to 0

MAPE	Mean Abs Percentage Error — scale-independent	Close to 0%
RMSRE	Root Mean Squared Relative Error	Close to 0
R² (r-squared)	% of variance explained by model (1=perfect)	Close to 1
Pearson r	Linear correlation between actual & predicted	Close to ± 1

Regression Code

```
import pandas as pd
import numpy as np
import math
from sklearn.linear_model import LinearRegression
from scipy.stats import pearsonr
import seaborn as sns

df = pd.read_excel('regdata.xlsx')

# Log transform to linearise relationship
df['logPrice'] = np.log(df['Price'])
df['logDemand'] = np.log(df['Dem'])

X = df[['logPrice']]
y = df['logDemand']

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

n = len(y)

# ---- PEARSON CORRELATION ----
corr, _ = pearsonr(df['logPrice'], df['logDemand'])
print(f'Pearson r      : {corr:.4f}')

# ---- MSE ----
mse = np.sum((y - y_pred)**2) / n
print(f'MSE           : {mse:.4f}')

# ---- RMSE ----
rmse = math.sqrt(mse)
print(f'RMSE           : {rmse:.4f}')

# ---- MAE ----
mae = np.sum(np.abs(y - y_pred)) / n
print(f'MAE           : {mae:.4f}')
```

```
# ---- MAPE ----
mape = 100 * np.sum(np.abs((y - y_pred) / y)) / n
print(f'MAPE          : {mape:.4f}%')

# ---- RMSRE ----
rmsre = math.sqrt(np.sum(((y - y_pred) / y)**2) / n)
print(f'RMSRE         : {rmsre:.4f}')

# ---- R SQUARED ----
ss_res = np.sum((y - y_pred)**2)
ss_tot = np.sum((y - y.mean())**2)
r2 = 1 - (ss_res / ss_tot)
print(f'R-squared     : {r2:.4f}')
```

Aim

Apply K-Means clustering on the Iris dataset and evaluate it using Silhouette Score, Adjusted Rand Index, and Mutual Information.

Metrics Explained

Metric	What it measures	Range
Silhouette Score	Intrinsic: compactness vs separation of clusters	-1 to 1, higher=better
Adjusted Rand Index (ARI)	Extrinsic: similarity to ground truth labels	-1 to 1, 1=perfect
Normalized Mutual Info (NMI)	Extrinsic: shared info between clusters & true labels	0 to 1, 1=perfect

Full Code

```
from sklearn import datasets
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.metrics.cluster import adjusted_rand_score,
normalized_mutual_info_score
import pandas as pd
import matplotlib.pyplot as plt

iris = datasets.load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

# ---- K-MEANS CLUSTERING ----
kmeans = KMeans(n_clusters=3, random_state=42)
df['cluster'] = kmeans.fit_predict(df[['sepal length (cm)', 'target']])

# ---- VISUALIZE ----
plt.scatter(x=df['sepal length (cm)'], y=df['target'], c=df['cluster'],
cmap='viridis')
plt.xlabel('Sepal Length'); plt.ylabel('Target Class')
plt.title('K-Means Clustering on Iris'); plt.colorbar(); plt.show()

# ---- SILHOUETTE SCORE (Intrinsic) ----
```

```

sil = silhouette_score(df[['sepal length (cm)', 'target']], df['cluster'])
print(f'Silhouette Score      : {sil:.4f}')
# Close to 1 = well-separated clusters

# ---- ADJUSTED RAND INDEX (Extrinsic) ----
ari = adjusted_rand_score(iris.target, df['cluster'])
print(f'Adjusted Rand Index   : {ari:.4f}')
# 1 = clusters perfectly match true labels

# ---- NORMALIZED MUTUAL INFO (Extrinsic) ----
nmi = normalized_mutual_info_score(iris.target, df['cluster'])
print(f'Normalized Mutual Info : {nmi:.4f}')
# 1 = clusters share maximum info with true labels

# ELBOW METHOD — find optimal k
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(iris.data)
    inertias.append(km.inertia_)
plt.plot(range(1, 11), inertias, marker='o')
plt.xlabel('Number of Clusters (k)'); plt.ylabel('Inertia')
plt.title('Elbow Method'); plt.show()
# Elbow = best k value

```

 **Tip:** Intrinsic metrics (like Silhouette) don't need true labels — useful in real-world unsupervised scenarios. Extrinsic metrics (ARI, NMI) require ground truth labels.

Aim

Implement time series forecasting using ARIMA on supermarket_sales data filtered by city.

Key Concepts

Stationarity	Time series should have constant mean and variance. ADF test checks this. $p\text{-value} < 0.05 \rightarrow$ stationary.
ADF Test	Augmented Dickey-Fuller — tests if the series has a unit root (non-stationary). Low p-value = stationary.
Decomposition	Breaks series into Trend + Seasonal + Residual components.
ACF	Auto-Correlation Function — how correlated is series with its past lags. Helps choose q (MA order).
PACF	Partial ACF — direct correlation at each lag. Helps choose p (AR order).
ARIMA(p,d,q)	p=AR order, d=differencing order for stationarity, q=MA order.
Resampling	<code>data.resample('D').mean()</code> — aggregates to daily frequency to reduce noise.

Full Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.arima.model import ARIMA
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error
from math import sqrt

df = pd.read_csv('supermarket_sales - Sheet1.csv')
df['Date'] = pd.to_datetime(df['Date'])

# ---- FILTER ONE CITY ----
data = df[df['City'] == 'Yangon'][['Date', 'Total']].copy()
data = data.sort_values('Date').set_index('Date')
```



```

# ---- PLOT RAW TIME SERIES ----
data.plot(figsize=(15,6))
plt.ylabel('Sales'); plt.xlabel('Date'); plt.title('Sales over Time'); plt.show()

# ---- ADF TEST (Stationarity) ----
result = adfuller(data['Total'])
print('ADF Statistic :', result[0])
print('p-value      :', result[1])
# p < 0.05 → stationary (no need to difference)

# ---- RESAMPLE TO DAILY ----
data_daily = data['Total'].resample('D').mean()

# ---- DECOMPOSITION ----
decomp = seasonal_decompose(data_daily.dropna(), model='multiplicative')
decomp.plot(); plt.show()

# ---- ACF and PACF ----
sm.graphics.tsa.plot_acf(data_daily.dropna(), lags=40); plt.show()
sm.graphics.tsa.plot_pacf(data_daily.dropna(), lags=40); plt.show()

# ---- TRAIN / TEST SPLIT ----
df_ts = data[['Total']].copy()
inputs = df_ts.index
target = df_ts['Total']
X_train, X_test, y_train, y_test = train_test_split(
    inputs, target, test_size=1/3, random_state=0)

# ---- ARIMA MODEL ----
# order=(p, d, q) - tune based on ACF/PACF
model = ARIMA(y_train, order=(0, 3, 1))
model_fit = model.fit()
print(model_fit.summary())

# ---- FORECAST ----
predictions = model_fit.forecast(len(y_test))

# ---- EVALUATION ----
mse = mean_squared_error(y_test, predictions)
mae = mean_absolute_error(y_test, predictions)
rmse = sqrt(mse)
print(f'MSE   : {mse:.4f}')
print(f'MAE   : {mae:.4f}')
print(f'RMSE  : {rmse:.4f}')

```

 **Note:** ARIMA order (p,d,q): p = lags in PACF before it cuts off, d = how many times you difference to make it stationary, q = lags in ACF before it cuts off.

Aim

Detect outliers in the Iris dataset using KNN (distance-based) and DBSCAN (density-based) methods.

Methods Compared

Aspect	KNN (Distance-Based)	DBSCAN (Density-Based)
Idea	Points far from their k nearest neighbours are outliers	Points in low-density regions (can't form cluster) are outliers
Key Parameter	n_neighbors, distance threshold	eps (neighbourhood radius), min_samples
Output	Outlier indices based on mean distance	Label = -1 means outlier
Best for	Numeric, low-dim data	Arbitrary shape clusters

Full Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

data = pd.read_csv('Iris.csv')
new_data = data.drop(['Id'], axis=1)

# ---- BOXPLOT to spot initial outliers ----
new_data.boxplot(); plt.title('Boxplot of Iris Features'); plt.show()

# =====
# METHOD 1: KNN Distance-Based
# =====
X = new_data.drop('Species', axis=1).values

nbrs = NearestNeighbors(n_neighbors=3)
nbrs.fit(X)
distances, indexes = nbrs.kneighbors(X)

# Plot mean distance per point
```

```

plt.plot(distances.mean(axis=1))
plt.title('Mean KNN Distances'); plt.ylabel('Distance'); plt.show()

# Points with high mean distance are outliers
threshold = 0.3 # Adjust visually from the plot
outlier_index = np.where(distances.mean(axis=1) > threshold)
outlier_values = new_data.iloc[outlier_index]
print('KNN Outliers:')
print(outlier_values)

# =====
# METHOD 2: DBSCAN Density-Based
# =====
df_db = data[['SepalLengthCm', 'SepalWidthCm']]

# eps = neighbourhood radius, min_samples = min points in neighbourhood
model = DBSCAN(eps=0.4, min_samples=10).fit(df_db)

colors = model.labels_
plt.scatter(df_db['SepalLengthCm'], df_db['SepalWidthCm'], c=colors,
            cmap='viridis')
plt.title('DBSCAN Clustering'); plt.show()

# Points with label = -1 are outliers
outliers_dbscan = data[model.labels_ == -1]
print('DBSCAN Outliers:')
print(outliers_dbscan)

```

 **Tip:** For the viva: DBSCAN marks label=-1 as noise/outliers. Tune eps by plotting k-distance graph — the 'elbow' gives you the eps value.

Aim

Use SMOTE (Synthetic Minority Oversampling TEchnique) to balance the Churn_Modelling dataset and improve classifier performance on the minority class.

What is Class Imbalance?

When one class has far more samples than another (e.g., 80% not churned vs 20% churned), a model tends to predict the majority class always. SMOTE synthetically generates new minority class samples by interpolating between existing minority points.

SMOTE vs No SMOTE — Expected Effect

Metric	Without SMOTE	With SMOTE
Majority class recall	Very high (~98%)	Slightly lower
Minority class recall	Very low (~5-20%)	Much higher
Overall accuracy	High but misleading	Slightly lower but fairer
F1 Score (minority)	Poor	Improved

Full Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE

df = pd.read_csv('Churn_Modelling.csv')

# ---- VISUALIZE IMBALANCE ----
data = df[['CreditScore', 'Age', 'Exited']]
sns.scatterplot(data=data, x='CreditScore', y='Age', hue='Exited')
plt.title('Before SMOTE'); plt.show()
```

```

print('Class distribution before SMOTE:')
print(df['Exited'].value_counts())

# ---- ENCODE CATEGORICAL COLUMNS ----
for col in df.columns:
    if df[col].dtype == 'O':
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])

# ---- TRAIN / TEST SPLIT (before SMOTE) ----
X = df.drop('Exited', axis=1)
y = df['Exited']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=101)

# ---- DECISION TREE WITHOUT SMOTE ----
print('\n--- Decision Tree WITHOUT SMOTE ---')
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
print(classification_report(y_test, clf.predict(X_test)))

# ---- APPLY SMOTE ----
smote = SMOTE(sampling_strategy='auto', k_neighbors=5, random_state=101)
X_os, y_os = smote.fit_resample(X_train, y_train)
print('Class distribution after SMOTE:', Counter(y_os))

# ---- DECISION TREE WITH SMOTE ----
print('\n--- Decision Tree WITH SMOTE ---')
clf.fit(X_os, y_os)
print(classification_report(y_test, clf.predict(X_test)))

# ---- LOGISTIC REGRESSION WITHOUT SMOTE ----
print('\n--- Logistic Regression WITHOUT SMOTE ---')
lr = LogisticRegression()
lr.fit(X_train, y_train)
print(classification_report(y_test, lr.predict(X_test)))

# ---- LOGISTIC REGRESSION WITH SMOTE ----
print('\n--- Logistic Regression WITH SMOTE ---')
lr.fit(X_os, y_os)
print(classification_report(y_test, lr.predict(X_test)))

# ---- VISUALIZE AFTER SMOTE ----
smote2 = SMOTE(random_state=101)
X_viz, y_viz = smote2.fit_resample(df[['CreditScore', 'Age']], df['Exited'])

```

```
df_over = pd.DataFrame(X_viz, columns=['CreditScore', 'Age'])
df_over['Exited'] = y_viz
sns.scatterplot(data=df_over, x='CreditScore', y='Age', hue='Exited')
plt.title('After SMOTE'); plt.show()

sns.countplot(data=df_over, x='Exited')
plt.title('Class Balance After SMOTE'); plt.show()
```

Aim

Perform inferential statistics on supermarket_sales data using one-sample Z-test, one-sample T-test, and two-sample tests.

When to Use Which Test

Test	When to use	Critical value ($\alpha=0.05$, right)
Z-Test (1-sample)	Sample size ≥ 30 , population std known	1.645
T-Test (1-sample)	Sample size < 30 , population std unknown	From t-table (df=n-1)
2-Sample T-Test	Compare means of 2 independent small groups	1.703 (df=27)
2-Sample Z-Test	Compare means of 2 independent large groups	1.645
ANOVA	Compare means across 3+ groups simultaneously	F-critical from table

Full Code

```
import pandas as pd
import math
import statistics
from scipy import stats

df = pd.read_csv('supermarket_sales - Sheet1.csv')

# H0: Mean total <= 250
# H1: Mean total > 250 (right-tailed test, alpha = 0.05)

# =====
# ONE-SAMPLE Z-TEST (n >= 30)
# =====
df_z = df.sample(n=100, random_state=42)
n_z = len(df_z)
x_bar_z = df_z['Total'].mean() # sample mean
meu = df['Total'].mean() # population mean
sigma = df['Total'].std() # population std dev
```



```

Z_SCORE = (x_bar_z - meu) / (sigma / math.sqrt(n_z))
print(f'Z Score: {Z_SCORE:.4f}')

critical_z = 1.645 # right-tail, alpha=0.05
if Z_SCORE > critical_z:
    print('Reject Null Hypothesis')
else:
    print('Do NOT Reject Null Hypothesis')

# =====
# ONE-SAMPLE T-TEST (n < 30)
# =====
df_t = df.sample(n=28, random_state=42)
n_t = len(df_t)
x_bar_t = df_t['Total'].mean()
s_t = df_t['Total'].std() # sample std dev

T_SCORE = (x_bar_t - meu) / (s_t / math.sqrt(n_t))
print(f'T Score: {T_SCORE:.4f}')

critical_t = 1.703 # df=27, alpha=0.05, right-tail
if T_SCORE > critical_t:
    print('Reject Null Hypothesis')
else:
    print('Do NOT Reject Null Hypothesis')

# =====
# TWO-SAMPLE INDEPENDENT T-TEST (Men vs Women Quantity)
# =====
men_arr = df[df['Gender'] == 'Male']['Quantity'].head(29).tolist()
women_arr = df[df['Gender'] == 'Female']['Quantity'].head(29).tolist()

men_mean = statistics.fmean(men_arr)
women_mean = statistics.fmean(women_arr)
men_std = statistics.stdev(men_arr)
women_std = statistics.stdev(women_arr)
men_len = len(men_arr)
women_len = len(women_arr)

# Pooled std dev formula
pooled_s = math.sqrt(((men_len-1)*men_std**2 + (women_len-1)*women_std**2) /
(men_len+women_len-2))
se = pooled_s * math.sqrt(1/men_len + 1/women_len)
T_2sample = abs(men_mean - women_mean) / se
print(f'2-Sample T Score: {T_2sample:.4f}')

df_2t = men_len + women_len - 2

```

```
critical_2t = 1.703 # df=27+29-2=54 approx, check table
if T_2sample > critical_2t:
    print('Reject Null Hypothesis')
else:
    print('Do NOT Reject Null Hypothesis')

# =====
# ANOVA - using scipy
# =====
# Test if mean Quantity differs across Product lines
groups = [df[df['Product line']==pl]['Quantity'].values
          for pl in df['Product line'].unique()]
f_stat, p_value = stats.f_oneway(*groups)
print(f'F-statistic: {f_stat:.4f}')
print(f'p-value      : {p_value:.4f}')
if p_value < 0.05:
    print('Reject H0 - at least one group mean is different')
else:
    print('Fail to Reject H0 - no significant difference')
```

 **Tip:** Decision rule: If test statistic > critical value → Reject H0. OR equivalently: if p-value < alpha (0.05) → Reject H0.

Aim

Demonstrate a complete Applied Data Science pipeline on any suitable dataset — from data loading to insights.

Suggested Pipeline Structure

Step	Stage	What to do
1	Load & Understand Data	<code>pd.read_csv()</code> , <code>df.shape</code> , <code>df.info()</code> , <code>df.head()</code>
2	Descriptive Statistics	<code>df.describe()</code> , mean, median, mode, skewness, kurtosis
3	Data Cleaning	Check nulls, impute (mean/median/mode), encode categoricals
4	Visualization	Histogram, boxplot, scatter, heatmap, pairplot
5	Outlier Detection	Boxplot IQR method or DBSCAN/KNN
6	Feature Engineering	Log transform, normalization, create new features if needed
7	Model Building	Train/test split, choose algorithm (classification or regression)
8	Evaluation	Accuracy/F1 (classification) or MSE/R ² (regression)
9	Insights	Summarize key findings and conclusions

Quick Cheat-Sheet Formulas

```
# CENTRAL TENDENCY
mean      = df['col'].mean()
median    = df['col'].median()
mode      = df['col'].mode()[0]
trimmed_mean = scipy.stats.trim_mean(df['col'], 0.1)

# SPREAD
variance  = df['col'].var()
std_dev   = df['col'].std()
sem       = df['col'].sem()
iqr       = df['col'].quantile(0.75) - df['col'].quantile(0.25)
```

```

# SHAPE
skewness      = df['col'].skew()
kurtosis      = df['col'].kurtosis()

# CLASSIFICATION METRICS (from TP, TN, FP, FN)
accuracy      = (TP + TN) / (TP + TN + FP + FN)
precision     = TP / (TP + FP)
recall/sens   = TP / (TP + FN)
specificity    = TN / (TN + FP)
f1_score      = 2 * precision * recall / (precision + recall)
fpr           = 1 - specificity

# REGRESSION METRICS
mse           = np.mean((y - y_pred)**2)
rmse          = np.sqrt(mse)
mae           = np.mean(np.abs(y - y_pred))
r2            = 1 - SS_res/SS_tot

# HYPOTHESIS TESTS
z_score       = (x_bar - mu) / (sigma / sqrt(n))
t_score       = (x_bar - mu) / (s / sqrt(n))
# Reject H0 if score > critical value

```

 **Note:** For Exp 10 case study: Pick diamonds.csv, titanic, or iris. Show all steps from EDA to model evaluation. A clean well-commented notebook gets full marks.

All the Best for Your Exam!

Remember: Read the question fully, choose the right dataset, and always add titles to your plots.